

Object Oriented Programming Concepts

Object Oriented Programming

Outline

2

- ❑ Operator overloading – overview
- ❑ Operator overloading - overloading basic operators with detailed examples.
- ❑ Operator overloading and Friend functions.

Operator Overloading

3

In C++, we can change the way operators work for user-defined types like objects and structures. This is known as **operator overloading**. For example,

Suppose we have created three objects `c1`, `c2` and `result` from a class named `Complex` that represents complex numbers.

Since operator overloading allows us to change how operators work, we can redefine how the `+` operator works and use it to add the complex numbers of `c1` and `c2` by writing the following code:

```
result = c1 + c2;
```

instead of something like

```
result = c1.addNumbers(c2);
```

This makes our code intuitive and easy to understand.

Note: We cannot use operator overloading for fundamental data types like `int`, `float`, `char` and so on.

Operator Overloading

4

Syntax for C++ Operator Overloading

To overload an operator, we use a special `operator` function. We define the function inside the class or structure whose objects/variables we want the overloaded operator to work with.

```
class className {  
    ... ..  
    public  
        returnType operator symbol (arguments) {  
            ... ..  
        }  
    ... ..  
};
```

Here,

- `returnType` is the return type of the function.
- `operator` is a keyword.
- `symbol` is the operator we want to overload. Like: `+`, `<`, `-`, `++`, etc.
- `arguments` is the arguments passed to the function.

Operator Overloading (Unary Operators)

5

Operator Overloading in Unary Operators

Unary operators operate on only one operand. The increment operator `++` and decrement operator `--` are examples of unary operators.

Example 1

```
int main() {  
    Count count1;  
  
    // Call the "void operator ++ ()" function  
    ++count1;  
  
    count1.display();  
    return 0;  
}
```

```
#include <iostream>  
using namespace std;  
class Count {  
    private:  
        int value;  
  
    public:  
        // Constructor to initialize count to 5  
        Count() : value(5) {}  
        // Overload ++ when used as prefix  
        void operator ++ () {  
            ++value;  
        }  
  
        void display() {  
            cout << "Count: " << value << endl;  
        }  
};
```

Operator Overloading (Unary Operators)

6

Output

```
Count: 6
```

Here, when we use `++count1;`, the `void operator ++ ()` is called. This increases the `value` attribute for the object `count1` by 1.

Note: When we overload operators, we can use it to work in any way we like. For example, we could have used `++` to increase `value` by 100.

However, this makes our code confusing and difficult to understand. It's our job as a programmer to use operator overloading properly and in a consistent and intuitive way.

Operator Overloading (Unary Operators)

7

The above example works only when `++` is used as a prefix. To make `++` work as a postfix we use this syntax.

```
void operator ++ (int) {  
    // code  
}
```

Notice the `int` inside the parentheses. It's the syntax used for using unary operators as postfix; it's not a function parameter.

Operator Overloading (Unary Operators)

8

Example 2

```
#include <iostream>
using namespace std;
class Count {
private:
    int value;
public:
    // Constructor to initialize count to 5
    Count() : value(5) {}
    // Overload ++ when used as prefix
    void operator ++ () { ++value;}
    // Overload ++ when used as postfix
    void operator ++ (int) {
        value++;
    }
    void display() {
        cout << "Count: " << value << endl;
    }
};
```

```
int main() {
    Count count1;

    // Call the "void operator ++ (int)" function
    count1++;
    count1.display();

    // Call the "void operator ++ ()" function
    ++count1;

    count1.display();
    return 0;
}
```

Output

```
Count: 6
Count: 7
```


Operator Overloading (Unary Operators)

9

The **Example 2** works when `++` is used as both prefix and postfix. However, it doesn't work if we try to do something like this:

```
Count count1, result;  
  
// Error  
result = ++count1;
```

This is because the return type of our operator function is `void`. We can solve this problem by making `Count` as the return type of the operator function.

```
// return Count when ++ used as prefix  
  
Count operator ++ () {  
    // code  
}  
  
// return Count when ++ used as postfix  
  
Count operator ++ (int) {  
    // code  
}
```

Operator Overloading (Unary Operators)

10

Example 3: Return Value from Operator Function (++ Operator)

```
#include <iostream>
using namespace std;
class Count {
private:
    int value;
public:
    // Constructor to initialize count to 5
    Count() : value(5) {}
    // Overload ++ when used as prefix
    Count operator ++ () {
        Count temp;
        // Here, value is the value attribute of the calling object
        temp.value = ++value;
        return temp;
    }
}
```

```
// Overload ++ when used as postfix
Count operator ++ (int) {
    Count temp;
    // Here, value is the value attribute of the calling object
    temp.value = value++;
    return temp;
}

void display() {cout << "Count: " << value << endl;}

};

int main() {
    Count count1, result;
    // Call the "Count operator ++ ()" function
    result = ++count1;
    result.display();
    // Call the "Count operator ++ (int)" function
    result = count1++;
    result.display();
    return 0;
}
```

Operator Overloading (Unary Operators)

11

Output

```
Count: 6  
Count: 6
```

Here, we have used the following code for prefix operator overloading:

```
// Overload ++ when used as prefix  
Count operator ++ () {  
    Count temp;  
  
    // Here, value is the value attribute of the calling object  
    temp.value = ++value;  
  
    return temp;  
}
```

Also, notice the code

```
temp.value = ++value;
```

The variable `value` belongs to the `count1` object in `main()` because `count1` is calling the function, while `temp.value` belongs to the `temp` object.

The code for the postfix operator overloading is also similar. Notice that we have created an object `temp` and returned its value to the operator function.

Operator Overloading (Binary Operators)

12

Operator Overloading in Binary Operators

Binary operators work on two operands. For example,

```
result = num + 9;
```

Here, `+` is a binary operator that works on the operands `num` and `9`.

When we overload the binary operator for user-defined types by using the code:

```
obj3 = obj1 + obj2;
```

The operator function is called using the `obj1` object and `obj2` is passed as an argument to the function.

Operator Overloading (Binary Operators)

13

```
class Complex {
private:
    float real;
    float imag;
public:
    // Constructor to initialize real and imag to 0
    Complex() : real(0), imag(0) {}
    void input() {
        cout << "Enter real and imaginary parts respectively: ";
        cin >> real;
        cin >> imag;
    }
    // Overload the + operator
    Complex operator + (const Complex& obj) {
        Complex temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }
}
```

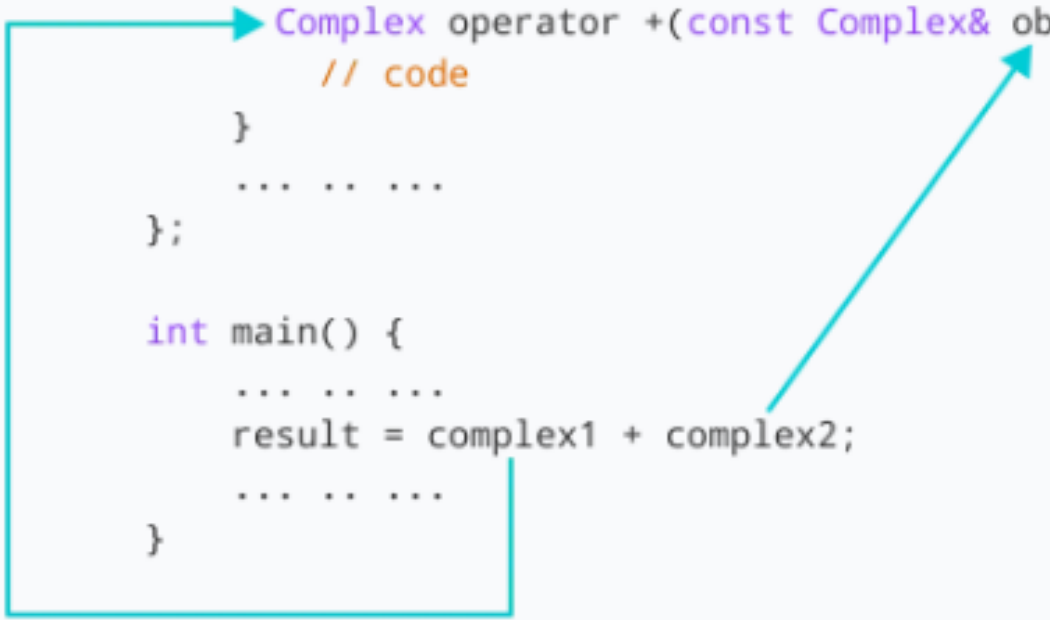
```
void output() {
    if (imag < 0)
        cout << "Output Complex number: " << real << imag << "i";
    else
        cout << "Output Complex number: " << real << "+" << imag
        << "i";
};

int main() {
    Complex complex1, complex2, result;
    cout << "Enter first complex number:\n";
    complex1.input();
    cout << "Enter second complex number:\n";
    complex2.input();
    // complex1 calls the operator function
    // complex2 is passed as an argument to the function
    result = complex1 + complex2;
    result.output();
    return 0;
}
```

Operator Overloading (Binary Operators)

14

```
class Complex {  
    ... ..  
public:  
    ... ..  
    Complex operator +(const Complex& obj) {  
        // code  
    }  
    ... ..  
};  
  
int main() {  
    ... ..  
    result = complex1 + complex2;  
    ... ..  
}
```



The diagram illustrates the resolution of the binary operator '+' in the main function. A teal line originates from the '+' operator in the expression 'complex1 + complex2' within the main function. This line extends to the left and then turns upwards, ending in an arrow that points to the 'Complex operator +' part of the overloaded function definition inside the Complex class. This visualizes how the compiler identifies the correct overloaded function to call based on the operands and the operator.

function call from complex1

Operator Overloading (Binary Operators)

15

Output

```
Enter first complex number:  
Enter real and imaginary parts respectively: 9 5  
Enter second complex number:  
Enter real and imaginary parts respectively: 7 6  
Output Complex number: 16+11i
```

In this program, the operator function is:

```
Complex operator + (const Complex& obj) {  
    // code  
}
```

Instead of this, we also could have written this function like:

```
Complex operator + (Complex obj) {  
    // code  
}
```

However,

- using `&` makes our code efficient by referencing the `complex2` object instead of making a duplicate object inside the operator function.
- using `const` is considered a good practice because it prevents the operator function from modifying `complex2`.

Operator Overloading (Binary Operators)

16

```
#include <iostream>
using namespace std;

class Complex{
public:
    float real;
    float img;

    Complex(){
        this->real = 0.0;
        this->img = 0.0;
    }

    Complex(float real, float img){
        this->real = real;
        this->img = img;
    }

    //overloading + operator
    Complex operator+(const Complex &obj){
        Complex temp;
        temp.img = this->img + obj.img;
        temp.real = this->real + obj.real;
        return temp;
    }
}
```

```
//overloading - operator
Complex operator-(const Complex &obj){
    Complex temp;
    temp.img = this->img - obj.img;
    temp.real = this->real - obj.real;
    return temp;
}

void display(){
    cout << this->real << " + " << this->img << "i" << endl;
}

};
```


Operator Overloading (Binary Operators)

17

```
int main()
{
    Complex a, b, c;

    cout << "Enter real and complex coefficient of the first complex number: " << endl;
    cin >> a.real;
    cin >> a.img;

    cout << "Enter real and complex coefficient of the second complex number: " << endl;
    cin >> b.real;
    cin >> b.img;

    cout << "Addition Result: ";
    c = a+b;
    c.display();

    cout << "Subtraction Result: ";
    c = a-b;
    c.display();

    return 0;
}
```

Conversion Operator

18

 **Conversion Operator:** We can also write conversion operators that can be used to convert one type to another type.

```
class Fraction
{
private:
    int num, den;
public:
    Fraction(int n, int d) { num = n; den = d; }

    // Conversion operator: return float value of fraction
    operator float() const {
        return float(num) / float(den);
    }
};

int main() {
    Fraction f(2, 5);
    float val = f;
    cout << val << '\n';
    return 0;
}
```

Output:

0.4

Conversion Constructor

19

```
class Point
{
private:
    int x, y;
public:
    Point(int i = 0, int j = 0) {
        x = i; y = j;
    }
    void print() {
        cout << "x = " << x << ", y = " << y << '\n';
    }
};

int main() {
    Point t(20, 20);
    t.print();
    t = 30;    // Member x of t becomes 30
    t.print();
    return 0;
}
```

Any constructor that can be called with a single argument works as a conversion constructor, means it can also be used for implicit conversion to the class being constructed.

Output:

```
x = 20, y = 20
x = 30, y = 0
```

Assignment Operator vs Copy Constructor

20

```
class Test {
public:
    Test() {}
    Test(const Test& t)
    {
        cout << "Copy constructor called " << endl;
    }

    Test& operator=(const Test& t)
    {
        cout << "Assignment operator called " << endl;
        return *this;
    }
};

// Driver code
int main()
{
    Test t1, t2;
    t2 = t1;
    Test t3 = t1;
    getchar();
    return 0;
}
```

Output

```
Assignment operator called
Copy constructor called
```

Explanation: Here, **t2 = t1;** calls the **assignment operator**, same as **t2.operator=(t1);** and **Test t3 = t1;** calls the **copy constructor**, same as **Test t3(t1);**

When should we write our own assignment operator in C++

21

The answer is same as Copy Constructor. If a class doesn't contain pointers, then there is no need to write assignment operator and copy constructor. The compiler creates a default copy constructor and assignment operators for every class. The compiler created copy constructor and assignment operator may not be sufficient when we have pointers or any run time allocation of resource like file handle, a network connection..etc. For example, consider the following program.

When should we write our own assignment operator in C++

22

```
// A class without user defined assignment operator
class Test
{
    int *ptr;
public:
    Test (int i = 0)    { ptr = new int(i); }
    void setValue (int i) { *ptr = i; }
    void print()        { cout << *ptr << endl; }
};

int main()
{
    Test t1(5);
    Test t2;
    t2 = t1;
    t1.setValue(10);
    t2.print();
    return 0;
}
```

Output of above program is "10". If we take a look at main(), we modified 't1' using setValue() function, but the changes are also reflected in object 't2'. This type of unexpected changes cause problems.

Since there is no user defined assignment operator in the above program, compiler creates a default assignment operator, which copies 'ptr' of right hand side to left hand side. So both 'ptr's start pointing to the same location.

We can handle the above problem in two ways.

- 1) Do not allow assignment of one object to other object. We can create our own dummy assignment operator and make it private.
- 2) Write your own assignment operator that does deep copy.

Same is true for Copy Constructor.

When should we write our own assignment operator in C++

23

```
#include<iostream>
using namespace std;

class Test
{
    int *ptr;
public:
    Test (int i = 0)    { ptr = new int(i); }
    void setValue (int i) { *ptr = i; }
    void print()        { cout << *ptr << endl; }
    Test & operator = (const Test &t);
};
```

Output

5

We should also add a copy constructor to the above class, so that the statements like "Test t3 = t4;" also don't cause any problem.

```
Test & Test::operator = (const Test &t)
{
    // Check for self assignment
    if(this != &t)
        *ptr = *(t.ptr);

    return *this;
}
```

```
int main()
{
    Test t1(5);
    Test t2;
    t2 = t1;
    t1.setValue(10);
    t2.print();
    return 0;
}
```

Assignment Operator vs Copy Constructor

24

Copy constructor

It is called when a new object is created from an existing object, as a copy of the existing object

It creates a separate memory block for the new object.

It is an overloaded constructor.

C++ compiler implicitly provides a copy constructor, if no copy constructor is defined in the class.

Assignment operator

This operator is called when an already initialized object is assigned a new value from another existing object.

It does not create a separate memory block or new memory space.

It is a bitwise operator.

A bitwise copy gets created, if the Assignment operator is not overloaded.

Operator Overloading and Friend Function

25

- Friend function using operator overloading offers better flexibility to the class.
- These functions are not a members of the class and they do not have 'this' pointer.
- When you overload a unary operator you have to pass one argument.
- When you overload a binary operator you have to pass two arguments.
- Friend function can access private members of a class directly.

Syntax:

```
friend return-type operator operator-symbol (Variable 1, Varibale2)
{
    //Statements;
}
```

Operator Overloading and Friend Function(Example1)

26

```
class Distance {
public:

    // Member Object
    int feet, inch;

    // No Parameter Constructor
    Distance()
    {
        this->feet = 0;
        this->inch = 0;
    }

    // Constructor to initialize the object's value
    // Parameterized Constructor
    Distance(int f, int i)
    {
        this->feet = f;
        this->inch = i;
    }
}
```

```
// Declaring friend function using friend keyword
friend Distance operator+(Distance&, Distance&);
};

// Implementing friend function with two parameters
Distance operator+(Distance& d1, Distance& d2) // Call by reference
{
    // Create an object to return
    Distance d3;

    // Perform addition of feet and inches
    d3.feet = d1.feet + d2.feet;
    d3.inch = d1.inch + d2.inch;

    // Return the resulting object
    return d3;
}
```

```
// Driver Code
int main()
{
    // Declaring and Initializing first object
    Distance d1(8, 9);

    // Declaring and Initializing second object
    Distance d2(10, 2);

    // Declaring third object
    Distance d3;

    // Use overloaded operator
    d3 = d1 + d2;

    // Display the result
    cout << "\nTotal Feet & Inches: " << d3.feet << " " << d3.inch;
    return 0;
}
```

Operator Overloading and Friend Function(Example2)

27

```
#include<iostream>
using namespace std;
class UnaryFriend
{
    int a=10;
    int b=20;
    int c=30;
public:
    void getvalues()
    {
        cout<<"Values of A, B & C\n";
        cout<<a<<"\n"<<b<<"\n"<<c<<"\n"<<endl;
    }

    void show()
    {
        cout<<a<<"\n"<<b<<"\n"<<c<<"\n"<<endl;
    }

    void friend operator-(UnaryFriend &x);    //Pass by reference
};
void operator-(UnaryFriend &x)
{
    x.a = -x.a;    //Object name must be used as it is a friend function
    x.b = -x.b;
    x.c = -x.c;
}

int main()
{
    UnaryFriend x1;
    x1.getvalues();
    cout<<"Before Overloading\n";
    x1.show();
    cout<<"After Overloading \n";
    -x1;
    x1.show();
    return 0;
}
```

Operator Overloading and Friend Function(Example2)

28

In the above program, **operator** – is overloaded using friend function. The **operator()** function is defined as a **Friend function**. The statement **-x1** invokes the **operator()** function. The object **x1** is created of class **UnaryFriend**. The object itself acts as a source and destination object. This can be accomplished by sending reference of an object. The object **x1** is a reference of object **x**. The values of object **x1** are replaced by itself by applying negation.

Output:

Values of A, B & C

10

20

30

Before Overloading

10

20

30

After Overloading

-10

-20

-30

Operator Overloading and Friend Function(Example3)

29

```
#include <iostream>
using namespace std;

class ThreeD {
    int x, y, z;
public:
    ThreeD() { x = y = z = 0; }
    ThreeD(int i, int j, int k) { x = i; y = j; z = k; }

    friend ThreeD operator+(ThreeD op1, int op2);
    friend ThreeD operator+(int op1, ThreeD op2);

    void show() ;
};

// This allows ThreeD + int
ThreeD operator+(ThreeD op1, int op2)
{
    ThreeD temp;

    temp.x = op1.x + op2;
    temp.y = op1.y + op2;
    temp.z = op1.z + op2;
    return temp;
}
```

```
// This allows int + ThreeD
ThreeD operator+(int op1, ThreeD op2)
{
    ThreeD temp;

    temp.x = op2.x + op1;
    temp.y = op2.y + op1;
    temp.z = op2.z + op1;
    return temp;
}

// Show X, Y, Z coordinates.
void ThreeD::show()
{
    cout << x << ", ";
    cout << y << ", ";
    cout << z << "\n";
}
```

```
int main()
{
    ThreeD a(1, 2, 3), b;

    cout << "Original value of a: ";
    a.show();

    cout << "\n";

    b = a + 10; // object + integer
    cout << "Value of b after b = a + 10: ";
    b.show();

    cout << "\n";

    b = 10 + a; // integer + object
    cout << "Value of b after b = 10 + a: ";
    b.show();

    return 0;
}
```

Important Points Operator Overloading

30

Implementing Operator Overloading

Operator overloading can be done by implementing a function which can be :

1. Member Function
2. Non-Member Function
3. Friend Function

Operator overloading function can be a member function if the Left operand is an Object of that class, but if the Left operand is different, then Operator overloading function must be a non-member function.

Operator overloading function can be made friend function if it needs access to the private and protected members of class.

Important Points Operator Overloading

31

Restrictions on Operator Overloading

Following are some restrictions to be kept in mind while implementing operator overloading.

1. Precedence and Associativity of an operator cannot be changed.
2. Arity (numbers of Operands) cannot be changed. Unary operator remains unary, binary remains binary etc.
3. No new operators can be created, only existing operators can be overloaded.
4. Cannot redefine the meaning of a procedure. You cannot change how integers are added.

Rules for Operator Overloading

32

Every programmer knows the concept of operation overloading in C++.

Although it looks simple to redefine the operators in operator overloading, there are certain restrictions and limitation in overloading the operators. Some of them are listed below:

1. Only existing operators can be overloaded. New operators cannot be overloaded.
2. The overloaded operator must have at least one operand that is of user defined type.
3. We cannot change the basic meaning of an operator. That is to say, We cannot redefine the plus(+) operator to subtract one value from the other.
4. Overloaded operators follow the syntax rules of the original operators. They cannot be overridden.

Rules for Operator Overloading

33

5. There are some operators that cannot be overloaded like size of operator(sizeof), membership operator(.), pointer to member operator(.*), scope resolution operator(::), conditional operators(?:) etc

6. We cannot use “friend” functions to overload certain operators. However, member function can be used to overload them. Friend Functions can not be used with assignment operator(=), function call operator(()), subscripting operator([]), class member access operator(->) etc.

7. Unary operators, overloaded by means of a member function, take no explicit arguments and return no explicit values, but, those overloaded by means of a friend function, take one reference argument (the object of the relevant class).

8. Binary operators overloaded through a member function take one explicit argument and those which are overloaded through a friend function take two explicit arguments.

9. When using binary operators overloaded through a member function, the left hand operand must be an object of the relevant class.

Rules for Operator Overloading

34

10. Binary arithmetic operators such as `+`, `-`, `*` and `/` must explicitly return a value. They must not attempt to change their own arguments.

Overloadable/Non-overloadable Operators

Following is the list of operators which can be overloaded:

<code>+</code>	<code>-</code>	<code>*</code>	<code>/</code>	<code>%</code>	<code>^</code>
<code>&</code>	<code> </code>	<code>~</code>	<code>!</code>	<code>,</code>	<code>=</code>
<code><</code>	<code>></code>	<code><=</code>	<code>>=</code>	<code>++</code>	<code>--</code>
<code><<</code>	<code>>></code>	<code>==</code>	<code>!=</code>	<code>&&</code>	<code> </code>
<code>+=</code>	<code>-=</code>	<code>/=</code>	<code>%=</code>	<code>^=</code>	<code>&=</code>
<code> =</code>	<code>*=</code>	<code><<=</code>	<code>>>=</code>	<code>[]</code>	<code>()</code>
<code>-></code>	<code>->*</code>	<code>new</code>	<code>new []</code>	<code>delete</code>	<code>delete []</code>

Following is the list of operators, which can not be overloaded:

<code>::</code>	<code>.*</code>	<code>.</code>	<code>?:</code>
-----------------	-----------------	----------------	-----------------

References

35

- <https://harshityadav95.medium.com/object-oriented-programming-c-d242601be045>
- <https://www.programiz.com/cpp-programming/operator-overloading>
- <https://www.geeksforgeeks.org/copy-constructor-vs-assignment-operator-in-c/>
- <https://www.tutorialride.com/cpp/friend-function-using-operator-overloading-in-c.htm>
- <https://www.geeksforgeeks.org/types-of-operator-overloading-in-c/>

THANK YOU